

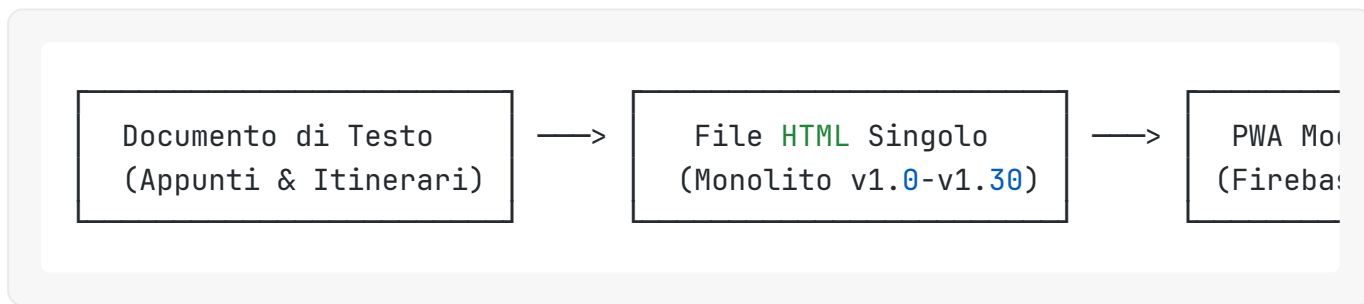
Quo Vadis — Manuale Tecnico e Funzionale

Versione Documento: 2.0 (Giugno 2026)

Autore: Manus AI

1. Storia Evolutiva del Progetto

Il progetto **Quo Vadis** nasce dall'esigenza di pianificare, tracciare e condividere un viaggio itinerante in furgone attraverso l'Europa, programmato per l'estate del 2026. L'evoluzione tecnologica dell'applicazione ha seguito un percorso incrementale, trasformandosi da un semplice archivio di appunti a una piattaforma web interattiva e resiliente.



Inizialmente, l'itinerario era strutturato come un **documento di testo non strutturato**, contenente elenchi di tappe, chilometraggi stimati e note sparse su attrazioni e campeggi [1]. Questa modalità si è rivelata presto limitata per la consultazione in mobilità e l'aggiornamento collaborativo.

Successivamente, il progetto è stato convertito in un **unico file HTML monolitico** (versioni da 1.0 a 1.30). Questa fase ha introdotto una prima interfaccia grafica basata su schede (tab) e una mappa interattiva Leaflet. Tuttavia, l'assenza di un database remoto rendeva impossibile il tracciamento in tempo reale, la sincronizzazione dei dati tra i dispositivi dell'equipaggio e l'utilizzo offline in aree a scarsa connettività.

La svolta è avvenuta con la transizione all'attuale architettura **Progressive Web App (PWA) Modulare (v1.60)**. Il codice è stato strutturato separando la logica applicativa principale (`app.js`), i dati statici dell'itinerario (`data.js` , `days-data.js`), la logica di rendering (`days-renderer.js`) e il ciclo di vita offline (`sw.js`). L'integrazione con **Firebase Realtime Database** e **Google Drive API** ha reso Quo Vadis un sistema di tracciamento e collaborazione completo.

2. Scopo e Numeri del Viaggio

Lo scopo principale di Quo Vadis è coordinare l’equipaggio, informare i familiari a casa in tempo reale e documentare l’esperienza giorno per giorno [1]. L’applicazione è dimensionata per gestire una spedizione stradale di grande portata attraverso il continente europeo, i cui parametri nominali sono riassunti nella tabella seguente:

Parametro	Valore Nominale	Dettagli e Copertura
Durata Totale	54 Giorni	Dal 26 Giugno al 18 Agosto 2026 [1]
Distanza Stimata	12.320 km	Percorso stradale complessivo pianificato [1]
Paesi Attraversati	13 Stati	 Italia,  Austria,  Rep. Ceca,  Polonia,  Lituania,  Lettonia,  Estonia,  Finlandia,  Norvegia,  Svezia,  Danimarca,  Germania,  Svizzera [2]
Equipaggio	2 Adulti + 2 Bambini	Organizzatori (Owners) e passeggeri [1]
Veicolo	Furgone Camperizzato	Mezzo di trasporto e pernottamento principale (autonomo con batterie LiFePo4 e gas) [1]
Evento Cardine	Capo Nord (Nordkapp)	Raggiungimento del punto più a nord (Giorno 23 - 19/07/2026) [1]
Evento Astronomico	Eclissi Totale di Sole	Prevista il 12/08/2026 a Palencia (Spagna) alle 20:29 locali [1]

3. Inventario delle Funzionalità dell’Applicazione

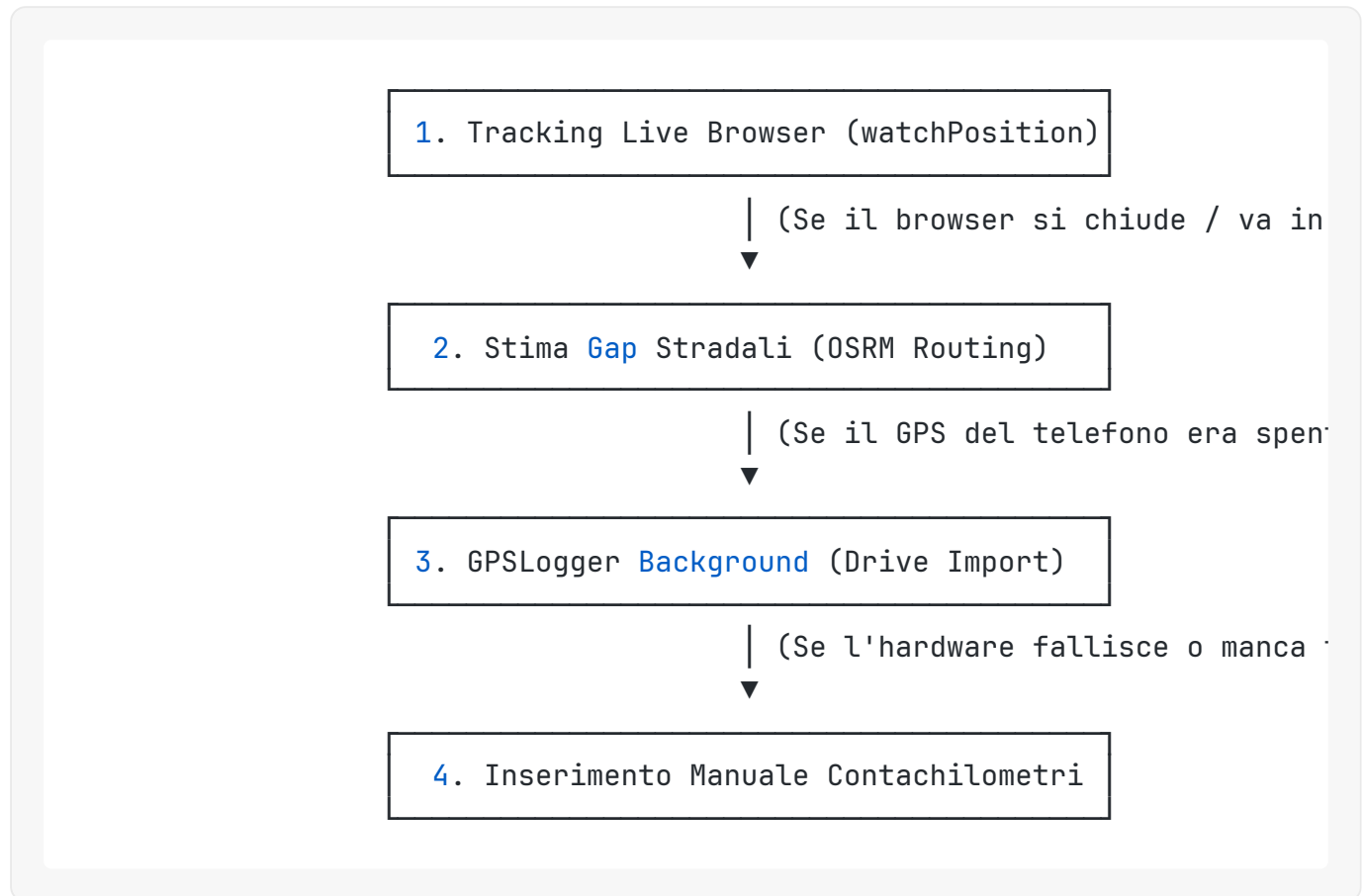
L’applicazione è organizzata in sezioni tematiche accessibili tramite una barra di navigazione inferiore. Ciascuna sezione risponde a una specifica esigenza logistica o informativa del viaggio:

- **Home (#tab-home)**: Rappresenta il cruscotto principale. Mostra un conto alla rovescia dinamico (espressi in giorni mancanti alla partenza) o, durante il viaggio, il progresso percentuale (es. “Giorno 15 di 54”) [3]. Include un widget meteo live che interroga l’API di Open-Meteo per la tappa odierna e le statistiche di viaggio aggregate (km percorsi, ore di guida, paesi visitati) [3].

- **Itinerario (#tab-giorni)**: Visualizza la lista cronologica dei 54 giorni [4]. Ogni giorno è espandibile e mostra la data, il tragitto stradale, i chilometri pianificati, le ore di guida stimate, le icone delle attività previste (es. 🎣 pesca, 🥾 trekking) e un link diretto a Google Maps per la navigazione [4].
 - **In Viaggio (#tab-posizione)**: È la sezione più dinamica. Contiene la mappa interattiva Leaflet che mostra la posizione in tempo reale del furgone (marker personalizzato a forma di furgone con orientamento basato sull'heading GPS), la traccia del percorso giornaliero e i marker dei parcheggi salvati [5]. Da qui l'organizzatore può avviare/fermare il tracciamento live, salvare un parcheggio con valutazione (stelle) e avviare la sincronizzazione manuale con Google Drive [5].
 - **Diario (#tab-diario)**: Funziona come un blog di viaggio collaborativo [6]. Permette di inserire note giornaliere, pensieri, "momenti top" (highlight) e caricare fotografie (salvate su Firebase Storage) [6]. A fine giornata, se l'utente non ha inserito nulla, l'app propone un widget di riepilogo automatico (Daily Recap) pre-compilato con i dati di viaggio della giornata [6].
 - **Cibo, Cultura, Attività, Luoghi**: Queste sezioni estraggono i dati in modo dinamico dal file `days-data.js` e li riorganizzano per categorie tematiche [7]:
 - *Cibo*: Piatti tipici, chioschi di street food consigliati (es. hot dog *Trekroneren* a Bergen) e ricette da cucinare in furgone [7].
 - *Cultura*: Musei, monumenti e attrazioni (con link automatici a Wikipedia per approfondimenti storici) [8].
 - *Attività*: Guide per trekking (difficoltà, dislivello, tracce GPX) e pesca (regolamenti nazionali e licenze) [4].
 - *Luoghi*: Punti di interesse naturalistici e urbani [8].
 - **Zaino (#tab-zaino)**: Una checklist interattiva per la preparazione dei bagagli, suddivisa in categorie (Documenti, Attrezzatura, Abbigliamento, Cucina, Farmaci) [9]. Lo stato di spunta di ogni oggetto è sincronizzato in tempo reale su Firebase con un sistema di debouncing per evitare scritture concorrenti [9].
 - **Chat (#tab-chat)**: Canale di comunicazione istantaneo interno all'app per l'equipaggio e i familiari autorizzati [10]. Supporta l'invio di messaggi di testo, condivisione di immagini, messaggi vocali registrati direttamente dal browser, risposte ai messaggi (reply-to) e reazioni con emoji [10].
-

4. Architettura del Tracking GPS a 4 Livelli

Il sistema di tracciamento della posizione e calcolo delle distanze è progettato per essere estremamente resiliente, garantendo l'integrità dei dati anche in assenza prolungata di rete o in caso di chiusura improvvisa dell'applicazione:



Livello 1: Tracking Live Browser

Quando l'applicazione è aperta in primo piano, la logica in `app.js` utilizza l'API nativa `navigator.geolocation.watchPosition()`. I punti GPS vengono catturati a intervalli regolari (10 secondi in modalità normale, 30 secondi in modalità risparmio energetico “Eco”) e inviati immediatamente a Firebase Realtime Database sotto il percorso `tracks/{data}/points`. Questo livello fornisce il movimento fluido del furgone sulla mappa per chi guarda il sito da casa.

Livello 2: Stima dei Gap Stradali via OSRM

I browser mobile tendono a sospendere l'esecuzione di JavaScript quando lo schermo viene spento o si passa a un'altra app (es. Google Maps per la navigazione). Per evitare linee rette irrealistiche sulla mappa (effetto “volo d'uccello”) al riavvio dell'app, Quo Vadis implementa un algoritmo di stima basato sul motore di routing stradale **OSRM (Open Source Routing Machine)**.

Se l'app rileva un salto temporale significativo (superiore a 5 minuti) e una distanza lineare maggiore di 100 metri dall'ultimo punto registrato, effettua una chiamata asincrona all'API OSRM:

```
var url = 'https://router.project-osrm.org/route/v1/driving/' + lastPt.lng
```

OSRM restituisce la reale geometria stradale del percorso più probabile tra i due punti. L'app calcola la distanza stradale effettiva, la somma ai chilometri giornalieri (`todayKm`) e inserisce i punti geometrici intermedi stimati nella traccia (marcandoli come `estimated: true`), facendo sì che la linea sulla mappa segua fedelmente le curve delle strade europee.

Livello 3: Rete di Sicurezza GPSLogger (Google Drive Import)

Per garantire una registrazione continua e professionale senza consumare la batteria del telefono con lo schermo sempre acceso, l'equipaggio utilizza l'applicazione Android open-source **Mendhak GPSLogger**. Questa app nativa gira come servizio di sistema a bassissimo consumo, registrando un punto GPS ogni 30 secondi e spegnendo il chip GPS tra un fix e l'altro [11].

Ogni 60 minuti (o alla pressione del tasto Stop), GPSLogger carica automaticamente il file `.gpx` aggiornato nella cartella Google Drive denominata `"GPSLogger for Android"` [11].

All'apertura di Quo Vadis, l'app controlla se è passata più di 1 ora dall'ultimo sync (parametro `DRIVE_SYNC_INTERVAL = 3600000`). Se l'intervallo è trascorso, l'app avvia un processo di sincronizzazione in background tramite le API di Google Drive:

1. Interroga la cartella `"GPSLogger for Android"` cercando file `.gpx` modificati negli ultimi 60 giorni.
2. Scarica i file GPX non ancora importati (tenendo traccia degli ID dei file in `localStorage`).
3. Analizza il contenuto XML del GPX, estrae i punti reali (latitudine, longitudine, velocità, timestamp) e li invia a Firebase.
4. Esegue un **merge intelligente con deduplicazione**: se un punto GPX ha un timestamp vicino (entro 30 secondi) a un punto già registrato dal tracking live del browser, l'app confronta l'accuratezza (`accuracy`) e conserva solo il punto più preciso. I punti stimati da OSRM (Livello 2) vengono automaticamente sostituiti dai punti reali di GPSLogger.

Livello 4: Inserimento Manuale Contachilometri

Il dato reale e inconfutabile della distanza percorsa è quello del contachilometri analogico del furgone. L'applicazione permette all'organizzatore di inserire manualmente questo valore a fine

giornata tramite il pannello “Modifica Km”. Il valore inserito viene salvato nel campo `odometerKm` dentro `dailySummaries/{data}`. Nelle schermate di riepilogo e nel calcolo delle statistiche globali, la presenza di `odometerKm` esclude qualsiasi calcolo basato su coordinate GPS, garantendo precisione millimetrica nei report finanziari e logistici.

5. Modelli di Distribuzione: PWA vs APK Nativo (Android) vs iOS

La scelta di sviluppare Quo Vadis come Progressive Web App (PWA) anziché come applicazione nativa (APK per Android o pacchetto `.ipa` per iOS) è stata dettata da precise considerazioni tecniche e logistiche:

Caratteristica	PWA (Implementata)	APK Nativo (Android) / iOS
Distribuzione	Immediata via URL (GitHub Pages)	Store ufficiali (Google Play / App Store) con tempi di approvazione
Aggiornamenti	Automatici al ricaricamento della pagina	Richiede download e installazione di un aggiornamento dallo store
Supporto Offline	Completo tramite Service Worker (caching asset e dati statici)	Nativo tramite database locali
Tracciamento Background	Limitato dalle politiche di risparmio energetico dei browser mobile	Illimitato tramite servizi di background nativi e permessi hardware
Costi di Sviluppo	Singola codebase HTML/JS, nessun costo di licenza sviluppatore	Doppia codebase (Kotlin/Swift) o framework cross-platform, licenza Apple (\$99/anno)

Analisi dei Modelli per Quo Vadis

Progressive Web App (PWA)

Vantaggi: Consente l’installazione immediata sulla schermata home sia su Android che su iOS (tramite il menu “Condividi -> Aggiungi a Home” di Safari). Gli aggiornamenti del codice (es. correzione di bug o aggiunta di funzionalità durante il viaggio) vengono distribuiti istantaneamente a tutto l’equipaggio semplicemente effettuando il push su GitHub; il Service Worker rileva la nuova versione di `version.json`, scarica gli asset aggiornati in background e mostra un banner di notifica per ricaricare l’app.

Svantaggi: Su iOS, le PWA hanno limitazioni storiche sull'accesso persistente alla geolocalizzazione in background quando lo schermo è spento. Android è più permissivo, ma tende comunque a terminare i processi dei browser per risparmiare memoria.

APK Nativo (Android) / TWA (Trusted Web Activity)

Cosa comporta: È possibile impacchettare la PWA esistente in un APK nativo utilizzando strumenti come Bubblewrap o Capacitor. Questo consentirebbe di pubblicare l'app sul Google Play Store o distribuire direttamente il file APK.

Vantaggi: Accesso facilitato per utenti meno esperti, integrazione più profonda con il sistema operativo.

Svantaggi: Introduce la complessità della gestione delle chiavi di firma, la conformità alle linee guida di Google (che hanno recentemente rimosso molte app di tracciamento come lo stesso GPSLogger dal Play Store per motivi di privacy) e rallenta la distribuzione degli aggiornamenti urgenti.

iOS Nativo

Cosa comporta: Richiede la riscrittura dell'interfaccia in Swift/SwiftUI o l'uso di un wrapper Cordova/Capacitor, oltre all'iscrizione al programma Apple Developer.

Vantaggi: Notifiche push native più stabili, tracciamento GPS in background impeccabile tramite le API CoreLocation.

Svantaggi: Costi elevati, processo di review severo che potrebbe rifiutare un'applicazione privata ad uso familiare.

Conclusione Architettuale: L'accoppiata **PWA (per la visualizzazione e interazione) + GPSLogger Nativo (per il tracciamento in background su Android)** rappresenta il compromesso perfetto. Evita i costi e le barriere degli store, garantendo al contempo un tracciamento GPS robusto e consumi di batteria ridotti.

6. Sicurezza e Protezione dei Dati

Nonostante l'applicazione sia in gran parte accessibile pubblicamente per consentire ad amici e conoscenti di seguire l'itinerario, Quo Vadis implementa rigorose misure di sicurezza per proteggere le funzioni amministrative e i dati sensibili:

Sicurezza a Livello di Database (Firebase Security Rules)

L'accesso in scrittura e lettura al database Firebase Realtime è regolato da regole JSON che applicano il principio del minimo privilegio in base al ruolo dell'utente:

```

{
  "rules": {
    "trips": {
      "viaggio-europa-2026": {
        ".read": "true",
        "chat": {
          ".write": "auth ≠ null && root.child('trips/viaggio-europa-2026',
        },
        "live": {
          ".write": "auth ≠ null && auth.uid = 'DRIVER_UID_HERE'"
        },
        "parking": {
          ".write": "auth ≠ null && root.child('trips/viaggio-europa-2026',
        }
      }
    }
  }
}

```

- **Lettura Pubblica:** I dati storici delle tracce, i diari e le informazioni generali sono leggibili da chiunque per consentire la visualizzazione pubblica del viaggio.
- **Scrittura Vincolata:** Solo gli utenti autenticati e presenti nella lista `approvedUsers` possono scrivere nella chat. Solo gli utenti registrati come `owners` (gli organizzatori) possono modificare i chilometri, salvare i parcheggi, caricare foto nel diario o modificare lo zaino.
- **Tracciamento Live:** Solo l'UID del conducente principale ha i permessi di scrittura sul nodo `live/` per aggiornare la posizione in tempo reale del veicolo.

Protezione da Vulnerabilità Web (XSS e Sanificazione)

Dato che la chat e il diario consentono l'inserimento di testo libero da parte degli utenti, l'applicazione implementa difese attive contro gli attacchi di tipo **Cross-Site Scripting (XSS)**:

- **Sanificazione dell'Input:** Ogni stringa inserita dall'utente viene passata attraverso la funzione `escapeHtml()` prima di essere inserita nel DOM. Questa funzione converte i caratteri speciali (come `<`, `>`, `&`, `"`) nelle rispettive entità HTML sicure:


```
function escapeHtml(str) {  
  if (!str) return '';  
  return String(str)  
    .replace(/&/g, '&amp;')  
    .replace(/</g, '&lt;')  
    .replace(/>/g, '&gt;')  
    .replace(/"/g, '&quot;');  
}
```

- **Separazione dei Contenuti:** La logica di rendering della chat applica prima `escapeHtml()` sul testo del messaggio, e solo successivamente applica la funzione `linkify()` per convertire in modo sicuro gli URL testuali in tag `<a>` cliccabili. I nomi degli autori e i timestamp vengono inseriti esclusivamente tramite la proprietà `textContent` dei nodi DOM, impedendo l'esecuzione di script malevoli iniettati nei profili utente.

7. Tecniche di Ottimizzazione del Codice

L'applicazione è progettata per essere estremamente veloce e reattiva anche su dispositivi datati o sotto reti mobili instabili (roaming 3G/4G nelle aree remote scandinave):

Caching Intelligente (Service Worker)

Il Service Worker (`sw.js`) implementa tre diverse strategie di caching a seconda della tipologia di risorsa richiesta:

1. **Stale-While-Revalidate (Asset Proprietari):** Applicata a file HTML, CSS, JS locali e icone dell'applicazione. Al caricamento, l'app serve istantaneamente la versione memorizzata nella cache locale (caricamento in <100ms), avviando contemporaneamente una richiesta di rete in background per scaricare eventuali aggiornamenti e aggiornare la cache per l'avvio successivo.
2. **Cache-First (CDN e Librerie Esterne):** Applicata alle risorse caricate da CDN esterne (come i fogli di stile e gli script di Leaflet, i font di Google o le librerie Firebase). Essendo file statici e versionati, vengono cercati esclusivamente nella cache locale. La rete viene interrogata solo se la risorsa non è presente, risparmiando preziosi kilobyte di traffico dati.
3. **Network-Only (API e Stato Remoto):** Le richieste verso Firebase, l'API meteo di Open-Meteo e il file di controllo `version.json` bypassano completamente la cache del Service

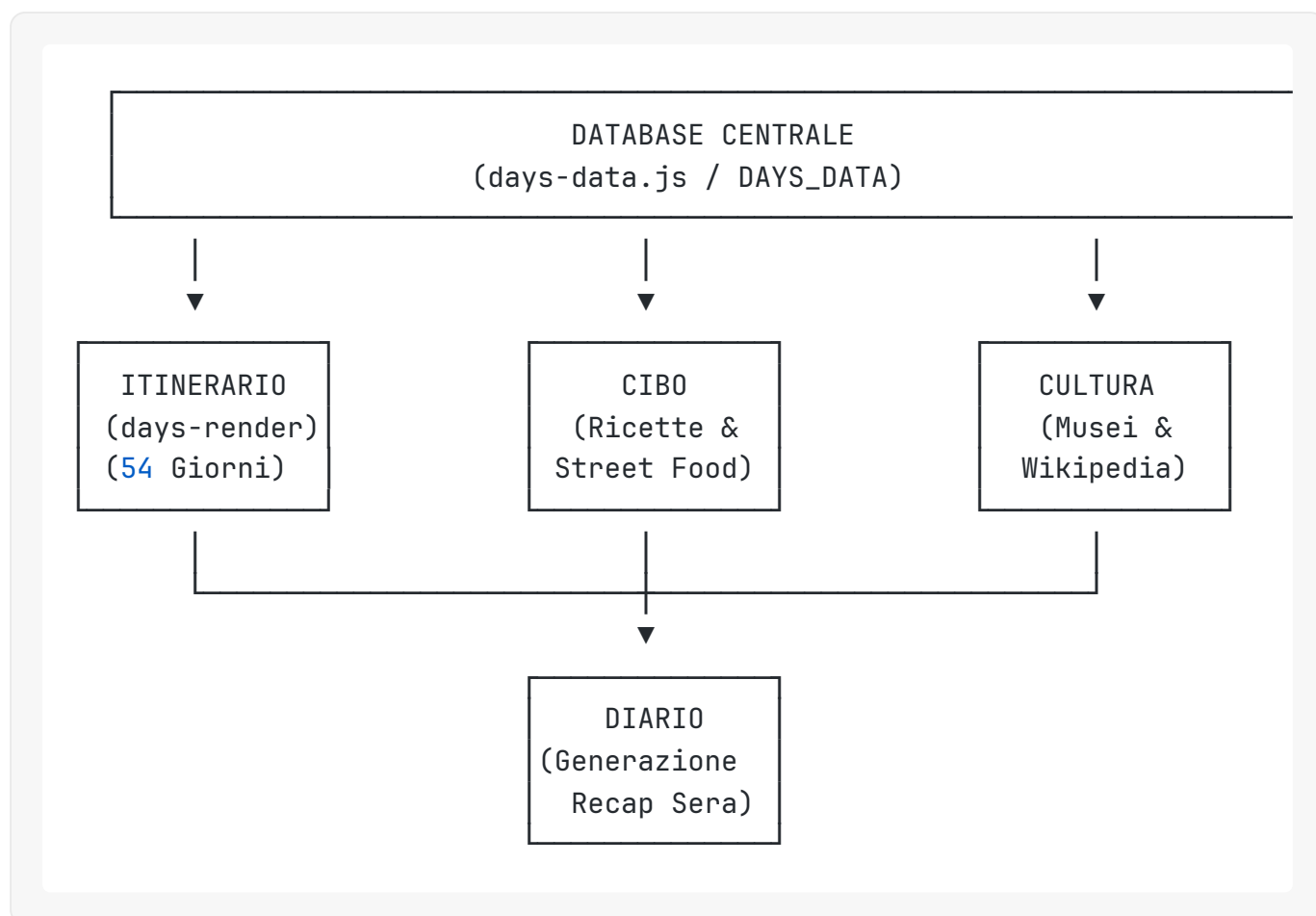
Worker per garantire che l'utente veda sempre i dati più aggiornati disponibili.

Ottimizzazione delle Prestazioni dell'Interfaccia (Rendering & Memory)

- **Lazy Loading delle Immagini:** Tutte le immagini caricate dagli utenti nel diario o nella chat utilizzano l'attributo nativo `loading="lazy"`. I file multimediali vengono scaricati dal server Firebase Storage solo quando l'utente scorre la pagina fino a renderli visibili sullo schermo, riducendo drasticamente il consumo iniziale di banda e memoria RAM.
- **Gestione della Memoria su Firebase (Listener Cleanup):** Per evitare memory leak e rallentamenti dovuti a listener attivi su schede non visibili, Quo Vadis implementa un registro centralizzato dei listener Firebase legato alla navigazione delle schede. Quando l'utente cambia tab (es. passa da "Chat" a "Zaino"), l'app esegue automaticamente la funzione `window.detachFirebaseListeners('chat')`, scollegando tutti i websocket attivi sul nodo della chat e riattivandoli solo quando l'utente torna su quella specifica sezione.
- **Debouncing delle Scritture:** Le modifiche alla checklist dello Zaino o lo stato di digitazione nella chat ("utente sta scrivendo...") utilizzano una funzione di debouncing. Se l'utente seleziona rapidamente più oggetti, l'app non invia una richiesta a Firebase per ogni singolo clic, ma attende un intervallo di inattività (es. 300ms) per inviare un unico payload cumulativo, riducendo il carico sul database e il consumo di batteria del telefono.

8. Analisi dei Flussi e Interconnessioni tra Sezioni

L'applicazione non è una semplice collezione di schede isolate, ma un ecosistema integrato in cui i dati fluiscono e si influenzano reciprocamente:



I dati fondamentali del viaggio risiedono nel file `days-data.js` sotto forma di un array strutturato di oggetti (`DAYS_DATA`). Questo file funge da “Single Source of Truth” (Sorgente Unica di Verità) per l'intero frontend:

- **Generazione dell'Itinerario:** Il modulo `days-renderer.js` cicla su `DAYS_DATA` per costruire dinamicamente l'interfaccia della scheda Itinerario. Calcola i totali progressivi dei chilometri e delle ore di guida e applica i filtri di ricerca testuale.
- **Estrazione Tematica (Cibo e Cultura):** Le schede “Cibo” e “Cultura” non possiedono un database proprio. All'avvio dell'applicazione, interrogano l'array `DAYS_DATA`, estraggono i nodi nidificati dedicati allo street food locale, alle specialità culinarie o ai monumenti storici, e li riorganizzano in liste filtrate per paese. Questo garantisce che se l'organizzatore modifica un ristorante consigliato nel Giorno 12 dell'itinerario, la scheda “Cibo” si aggiornerà automaticamente senza bisogno di interventi duplicati.
- **Integrazione con il Diario (Daily Recap):** A fine giornata, quando l'organizzatore apre la scheda Diario, l'applicazione confronta il giorno corrente del viaggio con `DAYS_DATA`. Se l'utente decide di scrivere il diario del giorno, il widget di riepilogo importa automaticamente i dati geografici del giorno (es. “Oggi siamo andati da Svolvær a Henningsvær”), i chilometri percorsi registrati dal GPS, le condizioni meteo storiche salvate e propone una bozza pre-compilata da integrare con foto e pensieri personali.

9. Risultati dell'Audit del Codice (Giugno 2026)

Durante l'audit statico e manuale del codice effettuato sulla versione 1.60 dell'applicazione, sono stati analizzati i file principali del progetto (`app.js` , `days-renderer.js` , `sw.js` e le Cloud Functions). Di seguito sono riassunti i risultati dell'analisi:

ESLint e Analisi Statica

L'analisi statica ha rilevato **2 errori** reali di tipo `no-func-assign` nel file `app.js` :

1. **Linea 7780**: Reassegnazione della funzione `sendMessage` .
2. **Linea 8257**: Reassegnazione della funzione `updateChatAuth` .

Diagnosi: Questi due casi non sono bug che bloccano l'esecuzione, bensì l'implementazione intenzionale di un **pattern decorator** (in cui la funzione originale viene avvolta in una nuova logica di autorizzazione prima dell'esecuzione). Tuttavia, per conformità agli standard moderni di sviluppo JavaScript ed evitare avvisi nei linter, si raccomanda di rifattorizzare queste righe utilizzando variabili di supporto distinte anziché riassegnare direttamente il nome della funzione dichiarata.

Sono stati inoltre rilevati circa 280 avvisi (warnings) relativi a variabili non dichiarate (`no-undef`). Quasi la totalità di questi è dovuta all'utilizzo di variabili globali condivise tra file diversi (es. variabili dichiarate in `data.js` e utilizzate in `app.js`) o a funzioni caricate da librerie esterne (come `firebase` o `L` per Leaflet) che non sono state esplicitamente dichiarate come globali nei commenti di intestazione dei file.

Analisi della Sicurezza e Robustezza

- **Prevenzione XSS**: Confermato l'utilizzo sistematico della sanificazione tramite `escapeHtml()` su tutti i contenuti testuali inseriti dagli utenti nella chat e nel diario prima del rendering.
 - **Gestione degli Errori**: Identificati alcuni blocchi `catch` vuoti nel modulo di sincronizzazione di Google Drive. Sebbene non causino crash dell'applicazione, si consiglia di aggiungere almeno un `console.warn()` per facilitare il debugging in caso di revoca improvvisa dei token OAuth2 da parte dell'utente.
-

10. Riferimenti e Risorse Esterne Utilizzate

Il corretto funzionamento dell'applicazione Quo Vadis è garantito dall'integrazione di servizi e librerie open-source di terze parti:

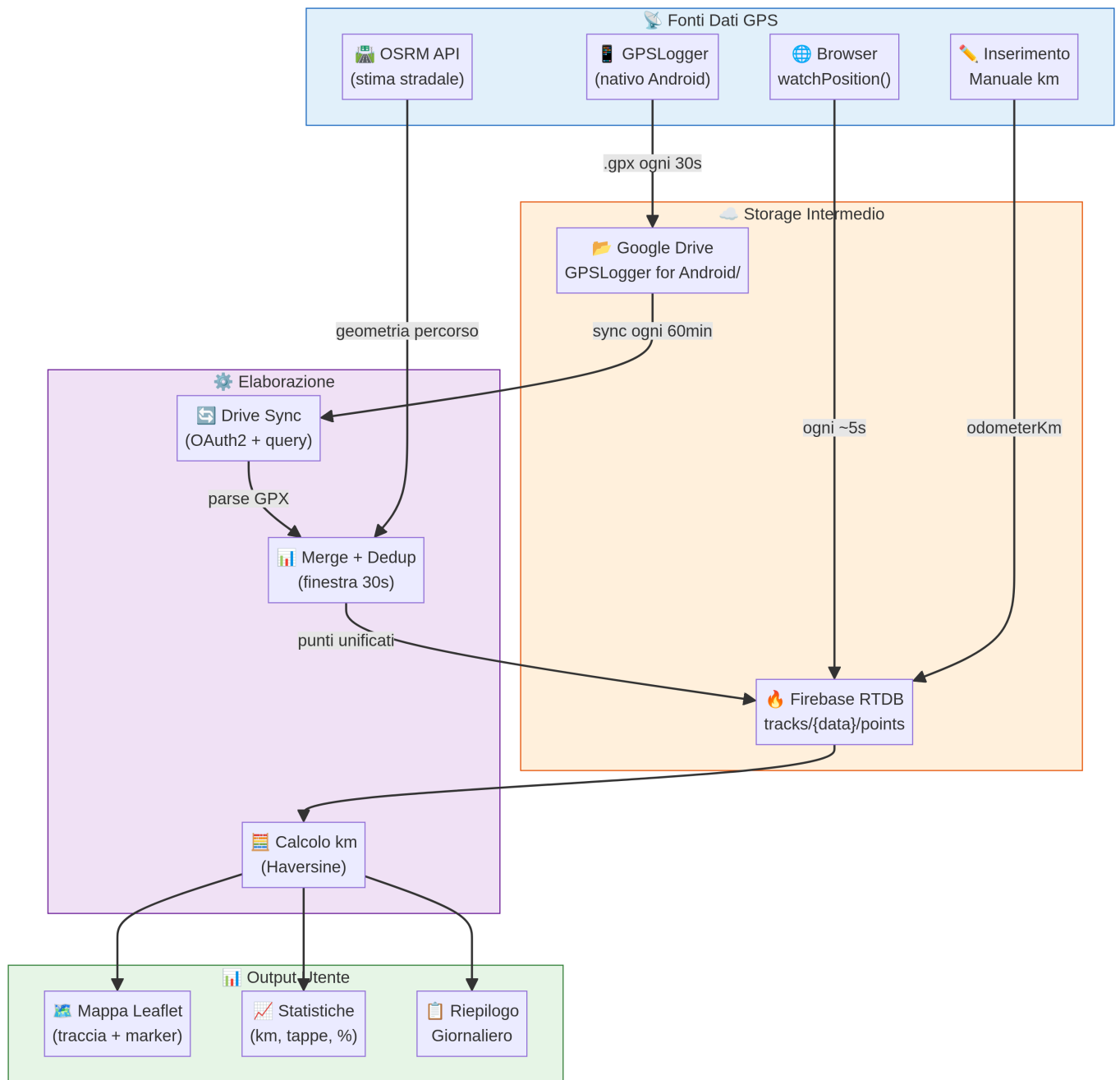
1. **Leaflet JS (v1.9.4)**: Libreria JavaScript utilizzata per la visualizzazione delle mappe interattive e la gestione dei layer geografici. <https://leafletjs.com/>
 2. **Firebase Web SDK (v10.12.2)**: Fornisce i moduli per l'autenticazione degli utenti, la sincronizzazione in tempo reale dei dati (Realtime Database) e l'archiviazione dei file multimediali (Firebase Storage). <https://firebase.google.com/>
 3. **Open-Meteo API**: Servizio meteorologico gratuito utilizzato per recuperare le previsioni del tempo in tempo reale e storiche per le coordinate geografiche di ciascuna tappa, senza richiedere chiavi API. <https://open-meteo.com/>
 4. **OSRM (Open Source Routing Machine)**: Motore di routing ad alte prestazioni basato sui dati di OpenStreetMap, utilizzato per calcolare i percorsi stradali reali e stimare i chilometri percorsi durante i buchi di ricezione GPS. <http://project-osrm.org/>
 5. **Nominatim OpenStreetMap**: Servizio di geocodifica inversa utilizzato per convertire le coordinate GPS (latitudine/longitudine) registrate dal furgone in nomi di località reali (città, nazione) mostrati nei widget dell'app. <https://nominatim.org/>
 6. **Wikipedia API**: Utilizzata per arricchire dinamicamente le schede culturali e naturalistiche dell'itinerario con informazioni storiche e immagini delle località attraversate. <https://www.wikipedia.org/>
-

11. Diagrammi di Architettura e Flusso Dati

Di seguito sono riportati i diagrammi tecnici che illustrano visivamente l'architettura e i flussi di dati dell'applicazione Quo Vadis.

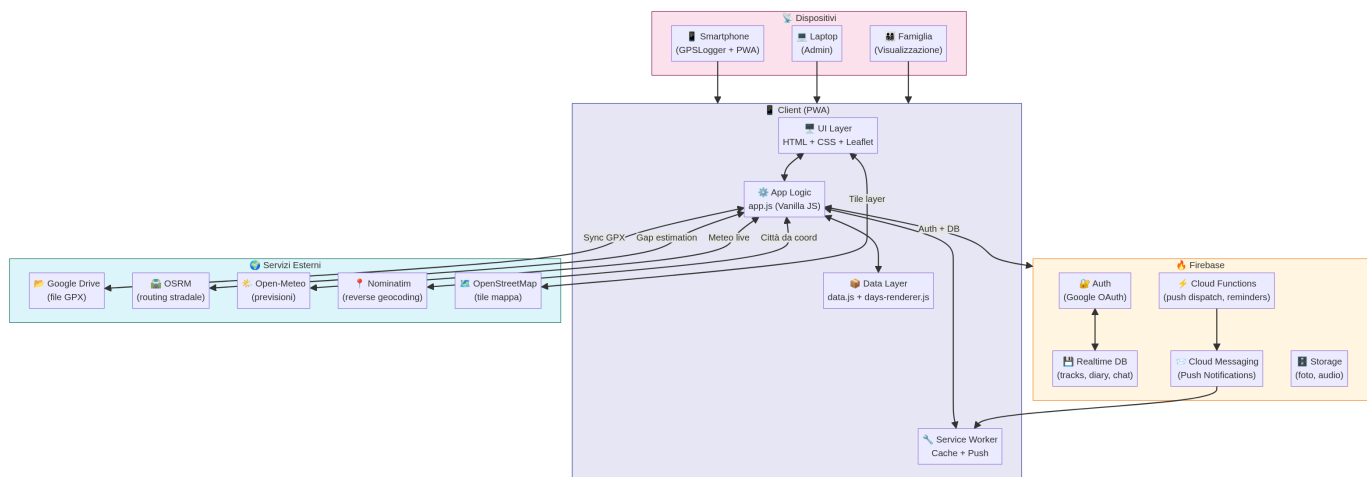
A. Flusso dei Dati di Tracciamento GPS

Questo diagramma mostra come i dati provenienti dalle quattro diverse fonti convergono nel database Firebase Realtime per poi essere visualizzati dall'utente finale sotto forma di mappe e statistiche.



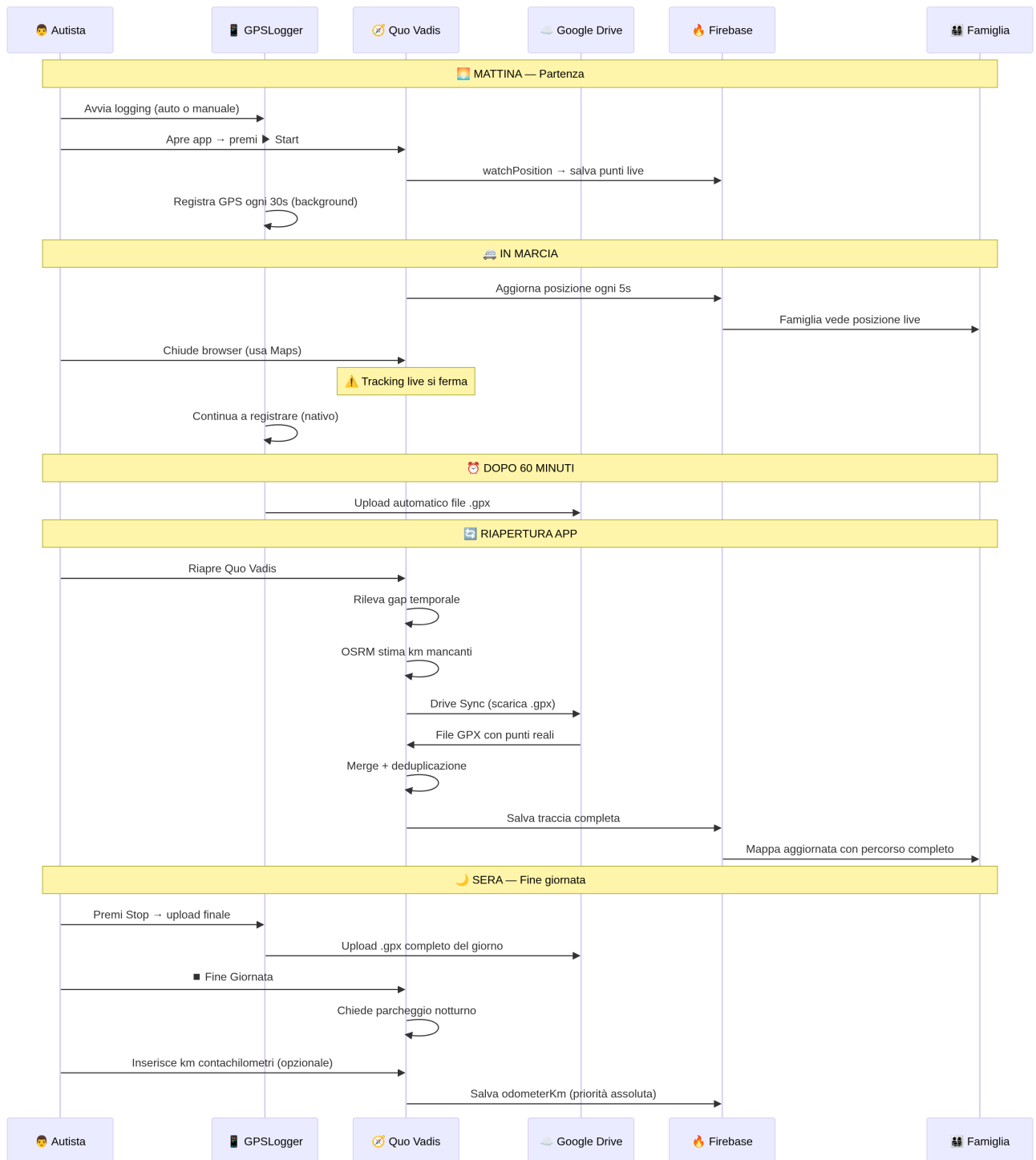
B. Architettura di Sistema Completa

L'applicazione è strutturata come una Progressive Web App (PWA) "serverless" che si interfaccia con diverse API esterne per mappe, meteo e geocodifica.



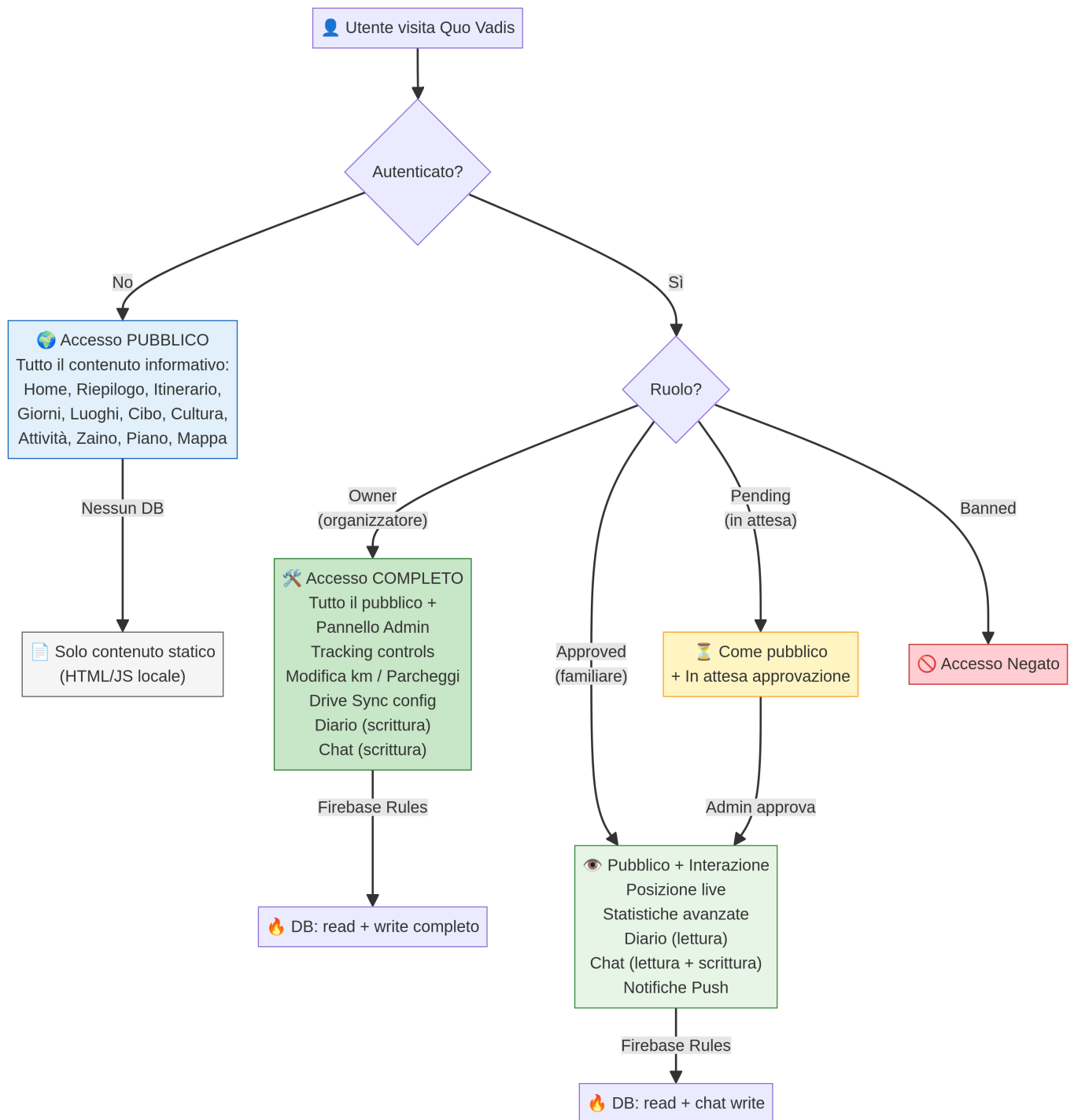
C. Ciclo di Vita Giornaliero del Tracking

Il flusso temporale tipico di una giornata di viaggio, dall'avvio mattutino fino al parcheggio serale.



D. Gestione dei Ruoli e Controllo Accessi

Il sistema di sicurezza basato su Firebase Authentication e Security Rules.



E. Sistema di Notifiche Push (FCM)

Il meccanismo di invio delle notifiche push tramite Cloud Functions e Firebase Cloud Messaging.

